AgentMPI: A Message-Passing Interface for Multi-Agent Harnesses

The AgentMPI authors
https://github.com/agentmpi/agentmpi

Abstract

Multi-agent systems built on large language models are written the way parallel programs were written before 1994: every framework bundles its own orchestration policy, its own communication mechanism, and its own failure handling, and none of them compose. The agent-interoperability protocols that have emerged since 2024—MCP, A2A, ACP—standardise transport, discovery and tool invocation, but they address a single client talking to a single server. None of them provides what a *parallel program* needs: group membership, collective operations, a consistency model for shared state, or defined semantics when a participant dies.

We argue that this is precisely the gap MPI filled for distributed-memory computing, and that MPI's answers largely transfer. We present AgentMPI, a message-passing protocol for building multi-agent harnesses. AgentMPI adopts MPI's load-bearing abstractions—communicators with isolated contexts, groups, typed envelopes, the full collective catalogue, one-sided windows, collective I/O, a name-shifted profiling interface, and ULFM-style revoke/shrink/agree—and changes exactly what the executor forces us to change.

Four properties of agents break MPI's assumptions, and each drives a specific protocol change. Context is a *cumulative* resource rather than a reusable buffer, so datatypes carry token bounds, receives are subject to admission control, and collective algorithm selection must reason about feasibility before cost; we show that a capacity-aware reduction tree must be planned against the root's cumulative ingest rather than its per-round ingest, a distinction with no MPI counterpart. Reduction operators are semantic rather than arithmetic, so operators declare commutativity, idempotency and an output bound, and the runtime refuses schedules those declarations do not support—we show that the natural choice of a commutative merge operator for a shared-terminology scan is *unsound*, because a commutative operator cannot express precedence. Failure is not fail-stop but a lattice of crashed, stalled, drifted and bankrupt, so the failure detector separates liveness from progress and recovery prefers rank replacement over shrinking. And because computation costs five to seven orders of magnitude more than communication, the latency–bandwidth trade-offs that shape MPI's algorithm selection invert: logarithmic schedules win everywhere.

We implement AgentMPI as a transport-independent runtime with a shared-directory device and a command-line binding, so that any process able to run a shell command—including a coding agent—is a rank. We evaluate it with genuine multi-agent runs: a twelve-rank book translation in which an exclusive parallel-prefix scan propagates terminology in $\lceil \log_2 p \rceil$ rounds instead of $p - 1$, an eight-module collaborative software build judged by a frozen integration suite, and fault-injection runs in which ranks are killed mid-collective. Microbenchmarks scale the protocol to 128 ranks.

1 Introduction

In April 1992, forty-odd people met in Williamsburg, Virginia, because distributed-memory parallel computing had a portability problem [25, 55]. Every vendor shipped a message-passing library—NX on Intel machines, Vertex on nCUBE, CMMD on the Connection Machine—and every research group shipped another—PVM, p4, Chameleon, Zipcode, PARMACS, Express, TCGMSG, CHIMP [32, 15, 52, 14]. The libraries were not merely different in spelling. They differed in what a message *was*, in whether a receive could match on a wildcard, in whether a collective operation existed at all. A parallel application was tied to a machine, and a parallel *library* was almost impossible to write, because a library and its caller shared one flat tag space and could silently steal each other's messages.

The MPI Forum's answer, standardised in MPI-1.0 in May 1994 [42, 26], was not a better library. It was a *specification*: a fixed vocabulary of communication operations with precise semantics, no implementation, and no opinion about what the ranks computed. Thirty years and five major revisions later, essentially every distributed-memory scientific code in production speaks it [10, 33].

1.1 The same problem, again

Multi-agent systems built on large language models are in the position distributed-memory computing occupied in 1992. There are many frameworks, each with its own model of what an agent is and how agents communicate. AutoGen rewrote itself around an actor model; LangGraph compiles a state machine over a shared, checkpointed state

object; CrewAI composes roles and tasks; the OpenAI Agents SDK passes control by handoff; MetaGPT and ChatDev encode a software-company org chart [45, 40, 21, 47, 35, 48]. A harness written for one is a rewrite for another, and the reason is familiar: each of them fuses three separable concerns—what the agents do, how they are scheduled, and how they exchange information—into one artifact.

The protocols that have appeared to address agent interoperability do not close this gap, and mostly do not try to. MCP standardises how one client reaches one server's tools, resources and prompts [46]. A2A standardises how one agent discovers and delegates a task to another, with agent cards, tasks and artifacts [1]. ACP and ANP occupy adjacent positions [36, 27]. All of them are, in the vocabulary of this paper, *point-to-point*: a client, a server, a request, a response. None of them has a notion of a group, a collective operation, a barrier, a consistency model for shared state, or a defined outcome when a participant dies mid-operation. They are the agent world's sockets. What is missing is its MPI.

That absence has a measurable cost. The largest empirical study of multi-agent LLM failures to date, which analyses 200+ traces across seven frameworks, finds that the majority of failures are not reasoning failures at all but *coordination* failures: agents that disobey a specification they were given, that lose track of what a peer already decided, that drop a step because they never learned another agent had finished, that terminate prematurely, or that never verify an artifact anyone produced [17]. Practitioners report the same thing from the other direction: Cognition's widely-read argument against multi-agent architectures is essentially that context does not propagate reliably between agents and that conflicting implicit decisions compound [56], while Anthropic's report on a production multi-agent research system spends most of its engineering discussion on exactly these problems—coordination, state, and recovery—rather than on prompting [7].

Those are not new problems. They are, respectively, contract violation, cache coherence, synchronisation, termination detection, and validation. A field that has been solving them for forty years has names for all of them.

1.2 What we take from MPI, and what we cannot

This paper asks what happens if the analogy is taken seriously and pushed as far as it will go. The answer is: much further than we expected, and then it breaks in four specific and interesting places.

What transfers is most of MPI's structure, and in particular its single best idea. A communicator is a group plus an opaque *context*, and the context guarantees that a message sent on one communicator cannot be received on another. That property is what made MPI *libraries* possible: before it, libraries reserved tag ranges by convention, and two libraries that disagreed produced silent, intermittent corruption [52, 33]. Multi-agent systems are in the pre-context era today—the dominant designs are a shared conversation buffer every participant reads and writes, or handoffs addressed by agent name—and they exhibit exactly the failure the Forum identified in 1993.

What does not transfer is the assumption that a rank is a cheap, deterministic, fail-stop process with reusable memory. Concretely:

1. **Context is consumed, not occupied.** An MPI rank may receive a terabyte over a job's life while never holding more than a megabyte; memory pressure is a function of the peak live set. Every token an agent reads is retained for the rest of the conversation, so pressure is a function of *cumulative ingest*. The right systems analogue is not `malloc` but a quota with admission control.
2. **Reduction operators are semantic.** `MPI_SUM` is associative, commutative, and output-size-preserving. "Merge these two drafts" is none of the three, and getting it wrong does not produce a rounding difference, it produces a different document.
3. **Failure is not fail-stop.** Agents crash, but they also stall while looking perfectly alive, return confidently malformed output, and run out of money. Conflating these produces the two classic pathologies of agent harnesses: killing a slow-but-working agent, and waiting forever for a fast-but-broken one.
4. **The cost model inverts.** In HPC a message costs microseconds and a floating-point operation costs subnanoseconds, so one optimises communication. An agent turn costs tens of seconds and real money, while a message is a file write. Communication is nearly free and computation is nearly everything.

Each of these forces a specific, minimal change to the MPI design rather than a wholesale redesign, and Sections 3 and 4 develop them in turn.

1.3 Contributions

- **A protocol.** AgentMPI (Section 4): communicators with isolated contexts, groups, token-bounded contract-carrying datatypes, four send modes including one that completes on *ingestion*, the full collective catalogue with declared operator algebra, one-sided windows with reference-by-default reads and a bounded retrieval read, collective artifact I/O, a failure lattice with revoke/shrink/agree/replace, a name-shifted profiling interface and a control/performance variable interface.
- **A cost model** (Section 5) in which the dominant term is agent turns on the critical path and feasibility is a hard constraint, and the demonstration that this inverts several of MPI's standard algorithm choices.
- **Two results about operator algebra** that we did not anticipate. A capacity-aware reduction tree must be planned against the root's *cumulative* ingest, not its per-round ingest, because an agent cannot free a buffer (Section 5); and a prefix scan over shared naming deci-

sions requires a *non-commutative* merge operator, because a commutative one cannot express precedence and therefore leaves participants disagreeing about what they agreed on (Section 4.4).
- **An implementation** (Section 6) with a swappable device layer and a command-line binding, so that any process able to run a shell command is a rank—which is what makes the protocol usable by agents that have no library bindings and never will.
- **An evaluation with genuine multi-agent runs** (Section 7): a twelve-rank book translation, an eight-module collaborative software build judged by a frozen integration suite, and fault-injection runs, together with microbenchmarks to 128 ranks.

We also report, in Section 8, a coordination failure we caused ourselves: a launcher concurrency limit interacting with a collective's all-participants requirement produced a deadlock that no rank could observe locally. It is a good advertisement for the protocol's own diagnostics, and a reminder that the hard part of a multi-agent system is not the agents.

2 Background: why MPI worked

MPI is often described as a library of communication functions. It is more useful, for our purposes, to read it as a set of decisions about *what to standardise and what to leave alone*. This section extracts the decisions we inherit. Readers who know MPI can skip to Section 3; readers who know agent systems and not MPI will find that most of what follows describes problems they have already met.

2.1 The standard is not an implementation

The Forum standardised an interface and refused to standardise behaviour below it [42]. `MPI_Bcast` says what every rank ends up holding; it says nothing about the schedule. MPICH and Open MPI between them implement at least five broadcast algorithms and choose among them at runtime by message size, communicator size and topology [54, 18]. Applications written in 1996 got faster in 2016 without being touched.

This is the decision that separates a protocol from a framework, and it is the one the agent ecosystem has not yet made. A harness written against AutoGen's group chat encodes a scheduling policy in the application; a harness written against `MPI_Bcast` encodes an intent.

2.2 Communicators, contexts, and safe composition

A communicator pairs an ordered group of ranks with an opaque *communication context*. Two properties follow. First, a rank is addressed by its index *within the communicator*, so a subgroup can be written as if it were the whole world. Second, and much more importantly, a message sent on one communicator can never be matched by a receive on another.

The second property is what made MPI libraries possible. Zipcode's designers had identified the problem directly: a library that posts a wildcard receive can consume a message the application sent to itself, and no discipline in tag allocation fixes it once two independently written libraries are involved [52]. `MPI_Comm_dup` makes the fix a one-liner: a library duplicates the communicator it is handed and is thereafter immune.

Multi-agent frameworks are, almost uniformly, back in the pre-context world. The shared conversation buffer is a flat tag space, and a sub-agent that reads "the last message" has no way to know whether it was addressed to it. This is not a hypothetical: it is one of the failure classes the MAST taxonomy names [17].

2.3 A small orthogonal core, plus a large convenience layer

MPI-1's point-to-point layer is, in principle, four calls. Everything else—the other three send modes, the nonblocking forms, the persistent requests, and every collective—could be written on top. The Forum included them anyway, for a reason that generalises: a collective operation *states an intent* that the implementation can exploit, while the hand-rolled equivalent states a schedule that it cannot. An `MPI_Allreduce` can be a recursive-doubling butterfly; a loop of sends and receives that computes the same thing cannot be.

2.4 Collective algorithms and their cost model

Collective algorithm design is where MPI's engineering depth lies. The standard cost model is Hockney's: a message of n bytes between two ranks costs $\alpha + n\beta$, with α the latency and β the reciprocal bandwidth [34]; LogP and its descendants refine it with overhead and gap terms [22, 4]. In this model, broadcast by binomial tree costs $\lceil \log_2 p \rceil (\alpha + n\beta)$ while van de Geijn's scatter-then-allgather costs roughly $(\log_2 p + p - 1)\alpha + 2\frac{p-1}{p} n\beta$, so the tree wins for small messages and the scatter/allgather form wins for large ones [54, 18]. Allreduce shows the same structure: recursive doubling costs $\log_2 p$ rounds but moves n bytes in each, while Rabenseifner's reduce-scatter-plus-allgather moves $2\frac{p-1}{p} n$ bytes in $2\log_2 p$ rounds [50]. The ring allreduce, later rediscovered by the deep-learning community, is bandwidth-optimal and latency-terrible: $2(p-1)$ rounds [49].

Every one of these trade-offs is a statement about the relative size of α and β. Section 5 shows what happens when the constants change by six orders of magnitude.

2.5 One-sided operations

MPI-2 added remote memory access because two-sided messaging forces the receiver to participate in every transfer, and there are algorithms where only the initiator knows what it needs [43]. A rank exposes a *window*; peers put, get and accumulate into it within synchronisation epochs—collective `MPI_Win_fence`, scoped post/start/complete/wait, or passive-target lock/unlock. MPI-3 tightened the memory model and added atomics and shared-memory windows [24].

The agent analogue is the shared scratchpad that every framework has and none of them specifies. What MPI-2 supplies and the scratchpads do not is epochs: a defined moment before which writes are not visible and after which they are. Without one, a reader sees a torn state and reasons confidently about half an answer.

2.6 Collective I/O

Naive parallel I/O collapses: a thousand ranks each writing a small unaligned piece of a shared file produce a thousand conflicting requests. ROMIO's two-phase I/O redistributes data to a small set of aggregators so that each issues one large contiguous write [53]. An agent harness writing to a shared repository has the same problem with a worse failure mode: twenty agents each committing a two-line edit produce merge conflicts, and a merge conflict costs an agent turn.

2.7 Fault tolerance: ULFM

MPI's position from 1994 until today is that after a process failure the state of MPI is undefined, with `MPI_ERRORS_ARE_FATAL` the default. On a machine whose mean time between failures is days and whose jobs checkpoint anyway, that was defensible. The User Level Failure Mitigation proposal adds the minimum viable set of primitives on top [13, 12]: `MPI_Comm_revoke` invalidates a communicator everywhere, so that a rank blocked on a peer that will never send gets an error instead of waiting forever; `MPI_Comm_shrink` builds a new communicator over the survivors and must be an agreement, since two ranks that disagreed about who died would build different worlds; and `MPI_Comm_agree` is a fault-tolerant agreement, which the FLP result tells us requires a failure detector [31]. ULFM's design principle—*the library should not recover for you, it should make recovery writable*—is one we adopt verbatim.

The earlier FT-MPI took a different line, offering a `REBUILD` mode that respawned failed ranks and preserved their indices [30]. For MPI that was expensive, because a rank's state is gigabytes of application memory. Section 4.7 argues that for agents it is the right default, because a rank's state is a prompt and a checkpoint.

2.8 Tooling: PMPI and MPI_T

MPI standardised a name-shifted profiling interface in 1994: every `MPI_X` has a weak symbol over `PMPI_X`, so a tool can interpose without touching any implementation. An entire ecosystem—mpiP, TAU, Score-P, Vampir, Jumpshot—exists because of that one decision [51, 38]. MPI-3 added `MPI_T` for reading and writing an implementation's internal control and performance variables [37]. Agent frameworks have nothing comparable; observability is per-framework and incompatible. We treat both as part of the protocol rather than as implementation features, for the same reason the Forum did.

2.9 The process manager is out of scope

Finally, MPI says almost nothing about how processes start. Launch and wire-up belong to a process manager speaking a separate protocol—PMI, and later PMIx, which exists precisely because exchanging endpoint information among 10^5 ranks through an argument list does not work [16]. The separation is why the same MPI library runs under Slurm, under a laptop, and under schedulers that did not exist when it was written. AgentMPI draws the same line, and Section 6 shows what lies on each side of it.

3 Why an agent is not a rank

The interesting part of this work is not the parts of MPI that transfer unchanged—those are numerous but unsurprising—but the four places where the executor's nature forces the design to differ. We state each mismatch precisely, because each one is the justification for a specific protocol feature in Section 4, and because a design that borrows MPI's vocabulary without acknowledging them would inherit the wrong semantics along with the right ones.

3.1 M1: context is consumed, not occupied

An MPI receive buffer is reusable. A rank can receive a terabyte over a job's lifetime while never holding more than a megabyte at once, because the memory occupied by message k is available again for message $k+1$. Memory pressure is a function of the *peak live set*, and the standard consequence is that a collective's memory cost is analysed per round.

Every token an agent reads enters its context and stays there. A rank that receives 200 messages of 5,000 tokens has spent a million tokens whether or not it needed two of them simultaneously. Pressure is a function of *cumulative ingest*. Three consequences follow, and all three are protocol-visible:

- A receive is not a data movement but an irreversible allocation against a quota, so it needs *admission control*:

the runtime must decide whether a payload may enter before materialising it (Section 4.3).

- Continuing past the quota requires rewriting history—replacing a span of consumed context with a smaller summary of it. This is a garbage collector whose liveness analysis is semantic, so the protocol supplies the mechanism and accounting and the harness supplies the policy.
- Collective feasibility becomes a first-class question. An MPI collective is always feasible; it may be slow, but no rank is *unable* to hold its share. `AMPI_Allgather` over p agents requires every rank to ingest $(p-1)s$ tokens by definition, so it stops working at $p > C/s$ regardless of algorithm. The runtime must be able to say so before the job spends anything.

There is a second-order effect that surprised us and that Section 5.3 develops: because ingest accumulates across *rounds*, the depth of a reduction tree is not free. An MPI rank that receives $k-1$ buffers per round reuses the same memory every round, so its peak is one round's worth and depth costs only latency. An agent root of a depth-d tree pays $(k-1)d$ contributions over the reduction's life. Planning a tree against the per-round figure produces schedules that look feasible, run two rounds, and then exhaust the root.

3.2 M2: reduction operators are semantic

`MPI_SUM` is associative, commutative, and output-size-preserving. Floating-point addition is not exactly associative, and MPI's community has long accepted the resulting last-bit non-reproducibility as the price of letting the implementation reassociate freely.

Agent operators fail all three properties, and the failures are not in the last bit:

- "Concatenate in rank order" is associative but not commutative.
- "Take a majority vote" is commutative but *not* associative: a pairwise tree of votes computes something different from a flat vote over the same inputs, and which answer wins depends on the number of ranks.
- "Summarise these two summaries" is approximately associative at best, and its output must be strictly smaller than its input or a tree of depth d produces an output of size 2^d.
- "Merge these two drafts" is none of the above.

So a protocol that wants to choose collective schedules on the application's behalf must be *told* the operator's algebra, and must refuse schedules the algebra does not license. `MPI_Op_create` already takes a `commute` flag for exactly this reason; AgentMPI needs that flag and two more—idempotency, which determines tolerance of duplicate delivery, and an output bound, which determines the legal tree depth (Section 4.4).

A related consequence: operator application is itself an agent turn. It costs seconds and money and can fail. Reductions therefore need retry, memoisation, and contract validation at every internal node—machinery that in MPI would be absurd and here is unavoidable.

3.3 M3: failure is not fail-stop

ULFM's model is binary: a rank is alive or it has failed, and failure is detected because the process is gone. Agents fail in at least four distinguishable ways, and the distinctions matter because the correct response differs:

Crashed
: the process is gone. Detected by absence of heartbeat; respond by replacing or shrinking.

Stalled
: the process is healthy and making no progress—looping on a tool call, retrying a rate-limited endpoint, or rereading its own output. Indistinguishable from a working agent by any liveness test, and indistinguishable from a *slow* agent without a progress signal.

Drifted
: the process returns output that violates the contract it was given: wrong schema, wrong language, a summary where a patch was expected. Perfectly alive, perfectly prompt, and useless. Detected only by validating content.

Bankrupt
: the process has exhausted its token or currency budget. Deterministic, predictable, and invisible to every failure detector in the distributed-systems literature, because no classical process runs out of money.

Conflating these produces the two pathologies every practitioner reports: killing a slow-but-working agent because it looked dead, and waiting forever for a fast-but-broken one because it looked alive. The protocol therefore needs two independent timeouts—liveness and progress—and a content-level error class alongside the transport-level ones.

There is also an asymmetry in the other direction, and it is favourable. Restarting an MPI rank is expensive: its state is the application's memory, which is why coordinated checkpointing and the entire literature on message-logging and the domino effect exist [28, 19]. An agent rank's recoverable state is its role, its checkpoint, and the messages it has acknowledged—kilobytes. So the balance between ULFM's *shrink* (cheap for the runtime, expensive for the application, whose data distribution was written against the old indices) and FT-MPI's *rebuild* (expensive for the runtime, free for the application) tips the other way, and Section 4.7 makes replacement the recommended path.

Finally, the classical recovery technique of message-log replay depends on the piecewise-deterministic assumption: re-executing a process from a checkpoint with the same message sequence reproduces its behaviour [6]. Agents violate this outright. Our substitute is content-addressed memoisation, which gives up bitwise reproduction and keeps the property that actually matters: a re-executed rank tells its peers the same thing it told them before.

3.4 M4: the cost model inverts

In HPC, $\alpha \approx 1\,\mu s$, $\beta^{-1} \approx 10\,\text{GB/s}$, and a floating-point operation costs a fraction of a nanosecond. Every collective algorithm in Section 2 is a trade between the α term and the β term, and computation is nowhere in the analysis because it is free relative to communication.

For agents: transmitting a message is a file write or an HTTP round trip—microseconds to milliseconds. Producing the message is an inference call: tens of seconds and, at current prices, cents to dollars. The ratio between the cost of moving a payload and the cost of creating it is between 10^5 and 10^7, and it points the opposite way.

Three consequences:

1. **Latency-optimal algorithms win essentially everywhere.** The HPC crossover at which bandwidth-optimal ring algorithms overtake logarithmic ones does not appear, because the bandwidth term is negligible. A ring allreduce over 64 agents costs 126 sequential turns; recursive doubling costs 6. Section 7.2 measures this.
2. **The right unit of cost is the turn, not the second.** Wall clock varies by model, provider and time of day; rounds on the critical path do not. We report both, and treat rounds as the invariant.
3. **Redundant computation is the thing to avoid.** Chatty protocols are fine; recomputing an agent's output is not. This is why memoisation is a protocol concern here and a curiosity in MPI.

3.5 What the mismatches do not change

It is worth being explicit about the negative result, because it is the paper's main structural claim. None of M1–M4 touches the matching semantics, the communicator/context construction, the group algebra, the topology interface, the collective *catalogue*, the epoch structure of one-sided operations, the two-phase I/O idea, the shape of revoke/shrink/agree, or the profiling interface. Those transfer verbatim. The mismatches bear on the type system, the operator algebra, algorithm selection, and the failure lattice—four well-localised places. A message-passing interface for agents is mostly MPI.

4 The AgentMPI protocol

AgentMPI specifies an interface and a set of semantics; it specifies neither an implementation nor a policy for what agents do. This section presents the protocol; Section 6 presents our reference implementation of it.

Names follow MPI's, deliberately: an operation that behaves like `MPI_Bcast` is called `AMPI_Bcast`, and one that behaves differently gets a different name. Table 1 summarises the surface.

4.1 Ranks, groups, communicators

A *rank* is an identity, not a process: an index within a communicator, plus a `RankSpec` recording role, model, provider, host, context store, capacity, price and tool permissions. MPI needs no such record because ranks are fungible; AgentMPI ranks are heterogeneous by construction, and both `AMPI_Comm_split_type` and cost-aware algorithm selection consult it.

Groups are local, ordered sets of ranks with the usual algebra (`incl`, `excl`, union, intersection, difference, translation) plus capability predicates. Keeping groups separate from communicators is worth more here than in MPI: a harness can retain the pre-failure membership and compute exactly which ranks were lost without any communication.

A communicator is a group plus a context, with MPI's guarantee: a message sent on one is never matched on another. `AMPI_Comm_dup` is the composition primitive of Section 2.2 and costs nothing—we derive the child context id deterministically from the parent's context and its collective counter, which is legitimate because the standard already requires collectives to be issued in a consistent order on every rank, and which removes a round trip from the most frequently used operation in any layered harness.

`AMPI_Comm_split_type` deserves a note. MPI-3 added `MPI_COMM_TYPE_SHARED` so a program could discover which ranks share physical memory and specialise. The agent analogue is the set of ranks sharing a resource with coherent cost: the same model (a handoff costs no re-grounding), the same provider quota (they contend), or the same context store (they can exchange references instead of content). Placing the chatty part of a computation inside such an island is the same optimisation.

4.2 Datatypes: contract and bound

An MPI datatype answers one question: how are these bytes laid out? That suffices because sender and receiver are the same deterministic program. An AgentMPI datatype answers three:

1. **Layout**: how is the payload rendered into a prompt?
2. **Contract**: what may the receiver assume, and how is that checked on arrival? A JSON type carries a schema; a patch type checks that it is a unified diff; user validators compose.
3. **Bound**: how many tokens may an instance cost, and how is an oversized instance reduced?

The third is the load-bearing one, and it exists because of M1. MPI's `count` and `datatype` let a receiver size a buffer, and an oversized message is a program error (`MPI_ERR_TRUNCATE`). In AgentMPI the receiver's buffer is its context window: small, permanently consumed, and shared with everything else the agent is doing. So the bound travels with the type, and the runtime is empowered to *reduce* an oversized payload rather than reject it. A type declares itself lossy or not; a lossy type carries a digest function; a non-lossy type that does not fit raises `AMPI_ERR_CONTEXT_OVERFLOW`.

Table 1: The AGENTMPI operation catalogue, with the MPI operation each derives from and the semantic change, if any. "—" means the semantics are MPI's, read over agent payloads.

AgentMPI	From	Change
Identity and groups		
`AMPI_Init`, `AMPI_Finalize`	`MPI_Init`	Idempotent and resumable; late join is normal, not exceptional.
`AMPI_Comm_rank`, `AMPI_Comm_size`, `AMPI_Comm_split`	same	—
`AMPI_Comm_dup`	same	Derives the new context id deterministically, so no extra round trip.
`AMPI_Comm_split_type`	`MPI_COMM_TYPE_SHARED`	Splits by shared *model*, provider, host or context store rather than shared memory.
`AMPI_Group_*`	same	Adds capability predicates; ranks are heterogeneous by construction.
`AMPI_Intercomm_create`	same	—
Types		
`AMPI_Type_contract`	—	New. Attaches a checkable schema; violation is a transport-level error class.
`AMPI_Type_bounded`	—	New. Attaches a token bound and a digest function.
`AMPI_Type_struct`, `AMPI_Type_contiguous`	same	Bounds compose.
Point to point		
`AMPI_Send/AMPI_Recv/AMPI_Isend/AMPI_Irecv`	same	Receive performs admission control and contract checking.
`AMPI_Ssend`	`MPI_Ssend`	Completes on *ingestion* by the peer, not on the receive having begun.
`AMPI_Bsend`, `AMPI_Rsend`, `AMPI_Mprobe/AMPI_Mrecv`	same	—
`AMPI_Psend_init/AMPI_Precv_init`	MPI-4 partitioned	—; maps onto incremental generation.
`AMPI_Waitsome`	same	Promoted: heavy-tailed turn times make it the default idiom.
Collectives		
`AMPI_Barrier`	same	Rootless by default; a rooted barrier makes the root a single point of failure at the worst moment.
`AMPI_Bcast`	same	Optional *refracting* relay: interior nodes may transform the payload for their subtree.
`AMPI_Scatter/AMPI_Scatterv/AMPI_Gather/AMPI_Gatherv`	same	Payloads travel by reference; materialisation is explicit.
`AMPI_Allgather`	same	—, but the runtime reports infeasibility, since peak ingest is $(p-1)s$ by definition.
`AMPI_Reduce/AMPI_Allreduce/AMPI_Reduce_scatter`	same	Operators declare algebra; illegal schedules are refused.
`AMPI_Scan/AMPI_Exscan`	same	—; the primitive that breaks carried dependencies.
`AMPI_Alltoall`, `AMPI_Neighbor_*`	same, MPI-3	—
One sided		
`AMPI_Win_create`, `AMPI_Put`, `AMPI_Accumulate`, `AMPI_Fetch_and_op`, `AMPI_Compare_and_swap`	same	—
`AMPI_Get`	`MPI_Get`	Returns a *reference*; materialisation is a separate, charged operation.
`AMPI_Win_query`	—	New. Bounded retrieval read, for windows larger than any reader.
`AMPI_Win_fence`, PSCW, lock/unlock	same	Locks are leased, so a dead holder cannot wedge the run.
I/O and faults		
`AMPI_File_write_at_all`	MPI-IO two-phase	—; aggregation eliminates merge conflicts rather than seek overhead.
`AMPI_Comm_revoke/AMPI_shrink/AMPI_agree`	ULFM	—
`AMPI_Comm_replace`	FT-MPI `REBUILD`	Promoted to the recommended path: rank state is kilobytes.
`AMPI_Checkpoint/AMPI_Restore`		Uncoordinated; the durable journal removes the domino effect.
Tools		
`PAMPI_*`	PMPI	—
`AMPI_T` cvars/pvars	`MPI_T`	Adds policy knobs (contract strictness, budgets) alongside performance knobs.

Making the bound part of the type—rather than a parameter of each call—is what lets the runtime predict a collective's peak ingest before any data exists, which is the precondition for capacity-aware algorithm selection (Section 5).

4.3 Point-to-point

Matching is MPI's, including the rules that carry the semantic weight: a receive matches on `(context, source, tag)` with wildcards for source and tag; **non-overtaking** holds between a pair on a communicator; unmatched arrivals and unmatched receives are queued. Three rules are added, all forced by the executor:

- **Deduplication.** Delivery is at-least-once—a retried send, a resurrected rank replaying its log—so a message is matched at most once per idempotency key.
- **Epoch filtering.** After a shrink, messages sent in a previous epoch by ranks that no longer exist must not match, or a recovered run consumes stale work.
- **Admission control.** A match completes only if the payload fits the receiver's budget; otherwise it is digested (if the type permits) or fails recoverably.

There is one further departure, in the non-overtaking rule itself. MPI can assume the network eventually delivers everything. AGENTMPI cannot: the rank that owes us sequence k may be dead, and blocking forever on a message with no sender is the failure mode we are trying to eliminate. The matching engine therefore holds a sequence gap for a bounded time and then advances past

it, recording the skip in the trace. This is a deliberate, visible weakening of a guarantee in exchange for liveness, and we prefer it stated in the protocol over discovered in production.

The four send modes are MPI's, but `AMPI_Ssend` is strictly stronger than `MPI_Ssend`. MPI's synchronous send completes when the matching receive has *begun*; AgentMPI's completes when the destination has *ingested* the message into its context. The stronger form is the useful one: a handoff protocol needs to know that a peer has been told something, not that a buffer was drained.

Partitioned communication (MPI-4) maps onto agents unusually well. A generating agent produces output incrementally; a consumer that waits for the whole document idles for the entire generation. `AMPI_Psend_init` plus `AMPI_Pready` lets the consumer start on section one while section two is being written, converting a serial dependency into a pipeline.

4.4 Collectives and the algebra of operators

The catalogue is MPI's. What is new is that an operator must declare its algebra, and that the runtime enforces it.

An `AMPI_Op` carries: `commute`, `associative`, `idempotent`, and `output_tokens`, an optional bound on its output size. From these the runtime derives which schedules are legal:

- A **non-associative** operator (a vote) may only be evaluated by a flat reduction. Reassociating it changes the answer, and a protocol whose answer depends on the rank count is not one anybody can build on. MPI silently reassociates and accepts a last-bit discrepancy; AgentMPI refuses, because the discrepancy is not in the last bit.
- A **non-contracting** operator (output no smaller than input) may not be used in a multi-level tree, whose peak ingest would grow with depth. The runtime says so, and names the fix.
- A **commutative, idempotent, contracting** operator admits any tree and tolerates duplicate delivery. Under at-least-once delivery this is the class to prefer, and the protocol makes that visible rather than folkloric.

Operators also come in an n-ary form, which is preferred when the inputs fit. This is not an optimisation but a quality decision: an agent asked to merge eight drafts at once does a better job than one asked to merge two at a time three times, because pairwise merging discards cross-cutting information at every step.

A commutative merge cannot express precedence. We expected the natural operator for reconciling shared naming decisions to be a union. It is not, and the reason is instructive. Consider a book translated by p agents that must agree on how to render each recurring proper noun. Expressed as an exclusive scan, chunk i receives the merged decisions of chunks $0..i-1$ and adopts them. With a commutative union operator, two chunks that independently propose different renderings for the same name both contribute, and a later chunk's prefix contains *both*—with no basis for choosing. Different ranks then see different prefixes, choose differently, and the disagreement the scan existed to remove survives it. We measured 0.82 declared consistency with a union operator where the protocol could have delivered 1.0.

The fix is a deliberately *non-commutative* operator, `AMPI_FIRST`: a key-wise merge in which the left operand wins. It is associative and idempotent, so every tree shape and any amount of duplicate delivery give the same result, but it is not commutative, and that is exactly what makes the prefix a function of rank order alone and therefore identical everywhere. With `AMPI_FIRST` the same experiment yields 1.0 (Section 7.3).

We state the general principle, because it applies to any shared convention an agent team must settle—schema choices, naming, file layout, API shape:

Design principle 1. A collective that exists to make participants *agree* requires an operator that can express precedence, and precedence is inexpressible in a commutative operator. Commutativity is the wrong default for consensus even though it is the right default for aggregation.

Refracting broadcast. In MPI an interior node of a broadcast tree forwards bytes verbatim. An agent interior node can also *adapt* the payload for its subtree: translate it, specialise it, compress it. `AMPI_Bcast` accepts an optional relay function, giving the same logarithmic schedule with a transformation at each level. This is strictly more expressive and strictly more dangerous, and the protocol makes the danger visible rather than preventing it: every envelope carries a provenance chain, so a receiver can see how many times its copy was rewritten.

4.5 One-sided operations: the blackboard

The most common complaint about multi-agent systems is that agents cannot see what other agents learned. Every framework's answer is a shared scratchpad, and every one of them is an unsynchronised shared-memory system: no epochs, no consistency model, no atomicity, no bound on growth. Those are the problems MPI-2's RMA chapter is about.

AgentMPI windows provide put, get, accumulate, get-accumulate, fetch-and-op and compare-and-swap, under the three MPI synchronisation regimes (collective fence, scoped post/start/complete/wait, passive-target lock/unlock). Atomicity is what makes a blackboard safe for concurrent writers, and it is what every hand-rolled scratchpad lacks: two agents that both append to a findings list with read-then-write will lose one of the findings, intermittently, and the harness will look like a model quality problem.

Two departures. First, `AMPI_Get` returns a *reference*, not content. An MPI `MPI_Get` of a megabyte costs microseconds and no lasting resource; an agent reading a megabyte has spent a third of its working life. Materialisation is a separate operation with a visible price, so a rank can hold a working set of hundreds of window entries for a few dozen tokens and pay only for what it reads. The window's *index*—every key with its size, version and owner, but no content—is correspondingly cheap, which means a rank can know what exists even when it could not possibly read it all.

Second, `AMPI_Win_query` has no MPI counterpart and is necessary for a structural reason: the window may be orders of magnitude larger than any participant's context, so "read the shared state" cannot mean "read all of it". The operation returns the highest-scoring entries that fit a token budget, together with an explicit list of what was omitted, so the caller knows it received a projection rather than the truth. We take this to be the correct systems framing of what the retrieval-augmented-generation literature has been building: not a way to make a model smarter, but the read operation of a shared memory whose readers are smaller than it is.

Locks are leased. An agent that dies holding a lock must not stop the run, and the lease expiry is recorded in the trace as a stolen lock—a far better outcome than an unexplained hang. Locking is per-key rather than per-window, because a whole-window lock serialises the entire team behind whichever agent is currently thinking, which can be minutes.

4.6 Collective I/O

`AMPI_File_write_at_all` is ROMIO's two-phase collective write with the motivation changed. Phase one redistributes: every rank ships its contribution to the aggregator responsible for the file. Phase two writes: the aggregator assembles the contributions in rank order and performs one atomic publish. In MPI this exists to turn many small unaligned writes into one large aligned one; here it exists so that exactly one writer touches each artifact and there is therefore no merge conflict to resolve. File views let a rank declare the region it owns, and overlapping views are reported at declaration time rather than discovered as a conflict later.

4.7 The failure model

The lattice of M3 is protocol-visible: a rank is `active`, `blocked`, `failed`, `stalled`, `drifted`, `bankrupt`, `evicted` or `finalized`, and the error classes follow—`AMPI_ERR_PROC_FAILED` and `AMPI_ERR_REVOKED` from ULFM, plus `AMPI_ERR_STALLED`, `AMPI_ERR_CONTRACT`, `AMPI_ERR_DRIFT` and `AMPI_ERR_BUDGET`.

Detection uses two independent signals. A *heartbeat* says the process exists; a *progress* report says it is getting somewhere. A detector built on the first alone cannot see a healthy infinite loop; one built on the second alone cannot distinguish a long turn from a dead process. Ranks that can run a background thread heartbeat continuously; ranks that cannot (an agent whose only interface is a shell, invoking the protocol once per operation) fall back to inferring liveness from activity, and the runtime reports which mode a rank is in so the harness can set the timeout accordingly rather than guessing. We regard this as an honest limitation rather than a solved problem, and Section 7.5 measures its cost.

Recovery is ULFM's, with one addition. `AMPI_Comm_revoke` invalidates a communicator everywhere, converting a deadlock into an error; it is the one thing an application genuinely cannot build for itself, because a rank blocked on a peer that will never send has no way to notice on its own. Our devices make propagation reliable for free: the revocation is a durable record, so it survives the revoker. `AMPI_Comm_agree` is a two-phase early-returning agreement whose decision is written with compare-and-swap, so a second decider cannot overwrite one that peers have already read. `AMPI_Comm_shrink` agrees on the survivor set first and *intersects* rather than unions the proposals, because a rank some survivors will not talk to must be treated as dead.

The addition is `AMPI_Comm_replace`: respawn the failed ranks, keeping their indices. A shrink renumbers everything, which is correct and painful, because the harness's data distribution—"rank i owns chapter i"—was written against the old indices. An agent rank's state is a role and a checkpoint, so replacement is cheap and preserves the distribution. The protocol does not know how to create an agent; `spawn` is supplied by the executor backend, exactly as MPI defers to a process manager.

Checkpoints are local and *uncoordinated*. Coordinated checkpointing exists because uncoordinated checkpointing plus message replay can cascade into the domino effect [19, 28], and that argument depends on replay being the only way to reconstruct a lost message. Here it is not: every message ever sent is durable and content-addressed in the journal, so a restarted rank reads what it missed instead of forcing peers to resend. Removing the need for coordination removes the domino effect with it, and is the largest simplification the durable-journal design buys.

4.8 Tools

AgentMPI specifies its profiling interface as part of the protocol. Every operation is routed through a profiler emitting a documented event stream—enter/leave states and matched send/receive arrows, following OTF2's and SLOG-2's model, with token and currency counters on every event [38]. That model is what makes Gantt timelines, communication matrices and critical-path analysis possible, and it transfers unchanged. `AMPI_T` exposes control variables (the eager limit in tokens, collective algorithm selection, contract strictness, budgets) and performance

variables (tokens sent and ingested, turns, rounds, digests, stalls, cache hit rate, spend). Both exist because the alternative—per framework, incompatible, bolted on—is where the field is now.

5 A cost model for agent collectives

Collective algorithm selection is an optimisation problem, and changing the objective changes the answer. This section states AgentMPI's objective, works out what it implies for the standard algorithms, and derives the one constraint that has no MPI analogue.

5.1 The model

MPI's working model is Hockney's: a point-to-point message of n bytes costs $\alpha + n\beta$ [34]. Computation is omitted because it is negligible. For agents we need three terms, and the one MPI omits dominates:

$$T \;=\; \gamma R \;+\; \alpha M \;+\; \beta N \tag{1}$$

where R is the number of **rounds** on the critical path (a round being one agent turn: read the inputs, think, emit), M the number of messages, N the number of tokens moved, and

- γ is the cost of one agent turn: 10–100 seconds and, at 2026 frontier-model prices, 10^{-2}–10^{0} currency units;
- α is the per-message transport cost: 10^{-4}–10^{-2} seconds for a filesystem or HTTP hop;
- β is the per-token transport cost, effectively 10^{-7} seconds.

So $\gamma/\alpha \in [10^3, 10^6]$ and $\gamma/(\beta \cdot 10^4) \approx 10^5$. Communication is free; turns are everything. Equation (1) is therefore, to within a rounding error,

$$T \approx \gamma R. \tag{2}$$

Two secondary objectives remain, and both are budgets rather than costs: total tokens ingested (which is money, and also context) and total turns (which is money). We report all three.

5.2 What $T \approx \gamma R$ does to algorithm selection

Table 2 gives round counts for the standard algorithms. Under Equation (2) the column that matters is the first, and the consequences are stark.

The bandwidth-optimal crossover disappears. In MPI, ring allreduce is preferred at large message sizes because it is bandwidth optimal: it moves $2\frac{p-1}{p}n$ bytes against recursive doubling's $n \log_2 p$ [50, 49]. Under Equation (2) that advantage is worth nothing, while its $2(p-1)$ rounds against $\log_2 p$ costs everything: at $p = 64$, 126 turns versus 6, or roughly twenty minutes versus one. There is no message size at which the ring wins, and Section 7.2 confirms the crossover does not appear.

Table 2: Rounds on the critical path and per-rank ingest, p ranks, contribution size s tokens. Under $T \approx \gamma R$ the leftmost column decides wall-clock and the rightmost decides feasibility.

Collective	Algorithm	Rounds	Peak ingest
Barrier	dissemination	$\lceil \log_2 p \rceil$	0
Bcast	flat	1	s
	binomial	$\lceil \log_2 p \rceil$	s
	chain	$p-1$	s
Gather	binomial	$\lceil \log_2 p \rceil$	$(p/2)s$ at root
Allgather	ring	$p-1$	$(p-1)s$
	Bruck	$\lceil \log_2 p \rceil$	$(p-1)s$
Reduce	flat	1	$(p-1)s$ at root
	k-ary tree	$\lceil \log_k p \rceil$	$(k-1)\lceil \log_k p \rceil\, m$
Allreduce	ring	$2(p-1)$	s
	rec. doubling	$\lceil \log_2 p \rceil$	$s \log_2 p$
Scan	chain	$p-1$	s
	Hillis–Steele	$\lceil \log_2 p \rceil$	$s \log_2 p$

Flat algorithms become attractive again. Symmetrically, a flat broadcast—the root sends $p-1$ messages—costs one round, against $\log_2 p$ for a tree. In MPI the flat form is dismissed because the root serialises $p-1$ sends, each costing α; here those sends are file writes and the root's cost is one turn regardless. The tree is preferable only when the root's *sending* is itself a turn (a refracting broadcast) or when message fan-out is rate-limited. Our decision function selects flat for $p \le 4$ and binomial above, and the honest statement is that the crossover is set by provider concurrency limits rather than by the cost model.

Scan is the highest-leverage collective. A harness with a carried dependency—each worker needs something every earlier worker decided—is usually written as a chain, at $p-1$ turns. As a prefix scan it is $\lceil \log_2 p \rceil$. At $p = 12$ that is 4 rounds instead of 11; at $p = 64$, 6 instead of 63. Nothing else in the catalogue offers that factor, and it applies to a structure (shared conventions established incrementally) that appears in almost every non-trivial agent workload.

5.3 Feasibility: the constraint MPI does not have

Every MPI collective is feasible. Under M1 (Section 3.1) that stops being true, and feasibility must be checked before cost.

Let C be a rank's usable context budget—its window less the reserve it needs for its own reasoning and output. Then:

Observation 1 (Allgather is intrinsically limited)**.**

`AMPI_Allgather` requires every rank to ingest $(p-1)s$ tokens, by definition and not by algorithm. It is therefore infeasible for $p > 1 + C/s$, and no choice of schedule helps. With $C = 80{,}000$ and $s = 2{,}000$, that is $p = 41$.

This is worth stating because "let every agent see every draft" is the first thing a harness author reaches for, and it has a hard ceiling that arrives sooner than intuition suggests. `AMPI_Allreduce` with a contracting operator is the scalable replacement, and the runtime says so by name when it predicts the overflow.

The reduction case is more subtle, and it is where we made a mistake worth recording. A k-ary reduction tree over p ranks has depth $d = \lceil \log_k p \rceil$, and each interior node receives $k-1$ contributions *per round*. In MPI the memory cost is one round's worth, $(k-1)m$, because the buffers are reused; depth costs only latency. We initially planned trees against that figure. It is wrong:

Observation 2 (Tree depth costs context)**.** An agent retains everything it reads, so the root of a depth-d k-ary tree ingests

$$I_{\text{root}} = (k-1)\, d\, m = (k-1)\lceil \log_k p \rceil\, m \tag{3}$$

tokens over the life of the reduction, where m is the operator's output bound. Planning against the per-round figure $(k-1)m$ yields schedules that look feasible, run two rounds, and exhaust the root.

Equation (3) is not monotone in k, which makes the planning problem genuinely non-trivial: increasing k raises the per-round fan-in but lowers the depth. The planner therefore searches $k \in [2, p]$ for the schedule minimising d subject to $I_{\text{root}} \le C$, breaking ties on I_{root}. A worked example: $p = 64$, $m = 1000$, $C = 10{,}000$. A degree-10 tree finishes in 2 rounds but costs the root $9 \times 2 \times 1000 = 18{,}000$ tokens—over budget. Degree 4 costs $3 \times 3 \times 1000 = 9{,}000$ in 3 rounds, and is selected. The protocol trades one round for feasibility, which is exactly the trade a harness author cannot make by hand because they cannot see the constant.

Finally, the operator's contraction is what makes any of this possible:

Observation 3 (Non-contracting operators forbid deep trees)**.** If the operator's output is no smaller than its inputs, m grows with depth, I_{root} grows super-linearly, and only a flat reduction is safe. A flat reduction costs the root $(p-1)s$, so the whole reduction is limited to $p \le 1 + C/s$ ranks.

This is why AgentMPI requires an operator to declare an output bound rather than inferring one: the declaration is what turns an unbounded reduction into a bounded one, and the runtime cannot discover it by observation without first running the thing it is trying to plan.

5.4 What the model does not capture

Three caveats, stated because we rely on the model in Section 7.

First, γ is not constant. An agent turn's duration grows with input length and output length, so a rank deep in a reduction tree, holding more context, is slower. The model treats γ as a constant and therefore under-predicts the cost of context-heavy schedules—in the direction that makes our capacity-aware planning look *less* valuable than it is.

Second, turn durations are heavy-tailed. A collective's cost is the *maximum* over participants, not the mean, so γ in Equation (1) should be read as a high quantile and the gap between mean and max widens with p. This is the standard tail-at-scale effect [23], and it is the argument for `AMPI_Waitsome` over `AMPI_Waitall` wherever the harness can proceed with partial results.

Third, the model says nothing about quality. A schedule that is cheaper may produce a worse answer—a deeper reduction tree summarises more aggressively and loses more—so cost and quality must be reported together. Section 7 does.

6 Reference implementation

AgentMPI is implemented in about 6,500 lines of dependency-free Python, with 260 tests. The architecture borrows from MPICH and Open MPI, for the reason those projects give: the layering is what lets the semantics outlive any particular transport.

6.1 Layering

Device interface
: The analogue of MPICH's Abstract Device Interface [9]. A device provides an at-least-once message channel, a content-addressed blob store, a key-value store with compare-and-swap, an append-only journal, and a clock. Notably it need *not* provide ordered delivery, connection state, or a broadcast medium—the assumptions that make MPI devices hard to port. Everything above the line is transport-independent, and the same test suite runs over every device.

Matching engine
: Posted-receive and unexpected queues, MPI's non-overtaking rule, deduplication, epoch filtering. Knows nothing about how bytes move; this is the separation MPICH's repeated ADI refactors argue for.

Communicators, collectives, RMA, I/O, fault tolerance
: The protocol proper, written against the matching engine.

Profiler
: Name-shifted interception of every operation, streaming to the journal rather than buffering—if a rank dies, a

buffered trace dies with it, and the failure you most wanted to see is the one you lose.

Two devices are implemented. The **journal** device uses a shared directory: message queues are directories, a send is one atomic `rename`, a receive is a directory listing, and only compare-and-swap needs exclusion. It requires nothing but a POSIX-ish filesystem, which is deliberate—the ranks of an AgentMPI job are frequently processes we do not control, each of which can run a shell command but cannot be handed a socket. A directory is the lowest common denominator, as TCP was for early MPI. The **in-proc** device is the Nemesis analogue: identical semantics, no syscalls, used for the test suite and for scaling studies where filesystem latency would obscure the protocol's own costs.

```
# rank 3 of a 13-rank translation job
export AMPI_ROOT=/runs/book AMPI_RANK=3

ampi bcast --root 0 --out spec.md
ampi scatter --root 0 --type json --out chunk.json
# ... read the chunk, propose terminology ...
ampi scan --exclusive --op ampi_first \
        --file proposal.json --type json --out prefix.json
# ... translate, adopting prefix.json ...
ampi gather --root 0 --file result.json --type json
ampi progress
```

Figure 1: A complete rank program. Every line is a shell command, so the executor can be a coding agent, a script, or a person. Errors are structured JSON on stderr with a machine-readable `error` field, so an agent can branch on `AMPI_ERR_TIMEOUT` versus `AMPI_ERR_REVOKED` without parsing prose.

6.2 Consumption, not polling, is what commits

One design point took us two attempts and is worth reporting, because it is forced by an assumption most message-passing runtimes do not have to question.

In a conventional runtime a rank is one process, so polling a message and matching it are separated by microseconds and it is harmless to treat the first as committing the second. An AgentMPI rank is frequently a *sequence* of short-lived processes—an agent that runs `ampi recv`, thinks for two minutes, then runs `ampi send`—and every one of them polls, matches at most one message, and exits. Our first implementation marked messages consumed at poll time, so a process that polled six messages and matched one silently destroyed the other five.

The fix is to separate the two: polling is speculative and in-process, consumption is explicit and durable. That in turn requires per-rank consumption state to survive across processes, and doing so naively—a set of consumed message identifiers—would grow without bound. We keep instead a per-`(context, source, destination)` watermark plus a small set of out-of-order exceptions above it, compacted after every consumption, which is $O(1)$ per peer in the common in-order case. The same durable state carries the send sequence counters and the collective counter, which is what lets a stateless endpoint sit in front of a stateful protocol.

6.3 The command-line binding

The binding that matters is not the Python one. `ampi` is a command-line tool exposing every operation, so that

> *anything that can run a shell command can be an AgentMPI rank.*

A rank's entire identity is two environment variables, `AMPI_ROOT` and `AMPI_RANK`; everything else—the group, the rank table, the peers' capabilities, the epoch—is discovered from the run directory at `AMPI_Init`, which is PMIx's design [16] scaled down. Blocking operations block, which is what an agent wants: `ampi recv` returns when the message arrives, and until then the agent is waiting at a shell prompt.

This is the same bet MPI made on having bindings for the languages people actually used. An agent protocol with only a Python binding is a Python framework; a rank implemented by a coding agent, a shell script and a Python program are indistinguishable on the wire and interoperate in one job.

6.4 The launcher is out of scope

AgentMPI specifies the wire-up and declines to specify the launch. A launch plan is a run directory plus one prompt per rank; how those prompts become running agents is the launcher's business. In our experiments the launcher is Cursor's subagent API; it could be containers, serverless functions, or people. Section 8 reports what happened when the launcher turned out to have a concurrency limit we had not accounted for.

6.5 Observability

Every operation emits enter/leave states and matched send/receive arrows, following OTF2's and SLOG-2's event model, with token and currency counters on every event. From that stream the tooling computes the standard MPI views—a communication matrix, a Gantt timeline, per-operation time breakdowns—plus two that are specific to this setting: cumulative context pressure per rank over time, and a critical path measured in *turns* rather than seconds. The turn-based critical path is the more useful number, because the ratio of total turns to critical-path turns is the harness's achievable parallelism and does not depend on how fast the model happened to be that day.

7 Evaluation

We ask four questions.

Q1 (Section 7.2)
Do the round counts and capacity bounds of Section 5 hold, and does the HPC bandwidth crossover really disappear?

Q2 (Section 7.3)
On a workload that looks embarrassingly parallel but carries a shared-convention dependency, does the prefix scan deliver consistency without serialising the job—with real agents?

Q3 (Section 7.4)
On a workload that is not parallel—eight mutually dependent modules of one program—do the protocol's structuring primitives produce a system that works?

Q4 (Section 7.5)
When ranks die mid-collective, does the job survive?

7.1 Methodology

Microbenchmarks use the in-process device with scripted executors, so they measure the protocol rather than the model; agent experiments use the shared-directory device with each rank driven by a coding agent invoking the `ampi` command-line tool. In every agent experiment, rank 0 runs a deterministic coordinator containing no model call, exactly as an MPI rank 0 is an ordinary rank that happens to own the input and output; every model call happens in ranks $1..p-1$.

We report rounds on the critical path as the primary cost measure and wall-clock as secondary, for the reason given in Section 5: rounds are exact and reproducible, wall-clock is one draw from a heavy-tailed distribution. Quality metrics are model-free and computed from the artifacts by a script that is the only thing permitted to write the paper's tables.

Two arms of the translation experiment had three of twelve translator ranks supplied by scripted stand-ins rather than agents, because of the launcher concurrency limit described in Section 8.1. The two arms use the *same* rank composition—agents at ranks $\{1, 2, 3, 5, 6, 7, 9, 10, 12\}$, stand-ins at $\{4, 8, 11\}$—so the comparison between them is matched, and we flag the caveat rather than hiding it.

7.2 Q1: microbenchmarks

Figure 2 reports rounds on the critical path from $p = 2$ to $p = 128$, measured from the trace rather than derived from a formula.

The measurements match the analysis exactly. Dissemination barrier, binomial broadcast, recursive-doubling allreduce and Hillis–Steele scan all show $\lceil \log_2 p \rceil$ rounds at every size, reaching 7 rounds at $p = 128$. The sequential chain scan shows $p - 1$: 7 rounds at $p = 8$, 31 at $p = 32$, 127 at $p = 128$. Ring allreduce shows $2(p - 1)$-behaviour, 31 rounds at $p = 32$ against recursive doubling's 5.

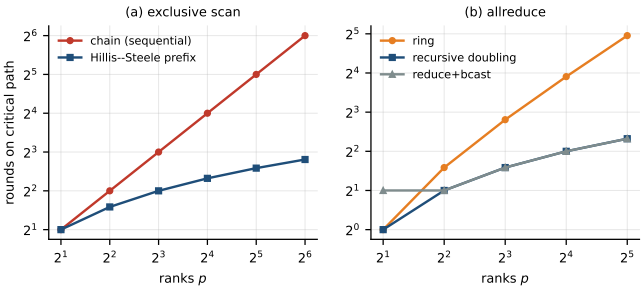

Figure 2: Rounds on the critical path, measured. (a) Exclusive scan: the sequential chain is $p - 1$ and the Hillis–Steele prefix is $\lceil \log_2 p \rceil$; at $p = 128$ that is 127 rounds against 7. (b) Allreduce: the ring's $2(p-1)$ rounds against recursive doubling's $\log_2 p$. In MPI the ring wins at large messages because it is bandwidth optimal; under $T \approx \gamma R$ it never wins.

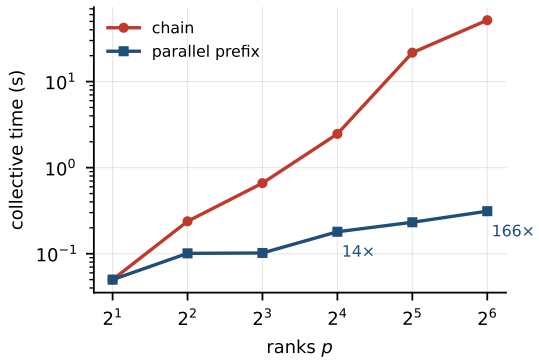

Figure 3: Measured exclusive-scan time, 50 ms synthetic think time per rank. Both axes logarithmic.

The crossover does not appear. Across the message sizes we tested, the ring never overtakes recursive doubling on wall-clock. This is the expected consequence of $\beta \to 0$ in Equation (1), and it is worth stating plainly because it retires a piece of received wisdom that a systems audience would otherwise import: *for agents, choose the latency-optimal collective algorithm unconditionally.*

Scan. Figure 3 gives measured collective time for the two scan schedules with a 50 ms synthetic think time. The advantage grows as predicted: 2.4× at $p = 4$, 6.5× at $p = 8$, 13.7× at $p = 16$, 93× at $p = 32$ and 166× at $p = 64$. The super-logarithmic growth beyond $p = 16$ is the tail effect of Section 5: the chain's cost is a sum of $p - 1$ heavy-tailed turn durations, so it grows faster than p while the prefix's is a maximum over $\log_2 p$ of them.

Feasibility. Figure 4 plots peak per-rank ingest against p for a 12,000-token budget and 1,000-token contributions. Broadcast is flat at s and never becomes infeasible. Allgather is $(p - 1)s$ and crosses the budget between $p = 8$ and $p = 16$; at $p = 128$ it demands 127,000 tokens per rank, more than ten times the budget, and no algorithm

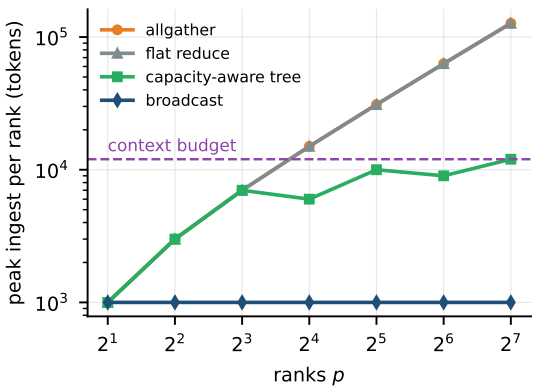

Figure 4: Peak per-rank ingest against p, 12,000-token budget, 1,000-token contributions. Allgather becomes infeasible between $p = 8$ and $p = 16$; the capacity-aware tree stays within budget to $p = 128$ by trading rounds for fan-in.

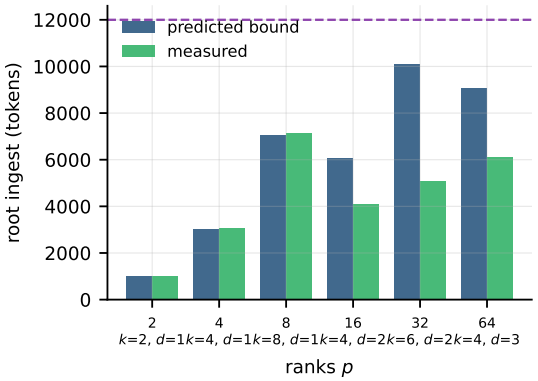

Figure 5: Predicted bound versus measured cumulative ingest at the root of a capacity-aware reduction tree. The dashed line is the budget. k is the selected degree, d the depth.

helps because the requirement is in the operation's definition. The capacity-aware reduction tree stays inside the budget at every size, at the cost of one extra round per doubling of the deficit: degree 8 in one round at $p = 8$, degree 4 in two rounds at $p = 16$, degree 4 in four rounds at $p = 128$.

The capacity model is sound. Figure 5 compares the predicted bound of Equation (3) against the root's measured cumulative ingest. The prediction is an upper bound at every size and is tight where the tree is full ($p = 2, 4, 8$: within 1.7%) and conservative where the last level is partial ($p = 16, 32, 64$: the root has fewer children than $k - 1$ in its final round). No measurement exceeds the bound, which is the property the planner needs.

Table 3: Book translation, twelve translator ranks over the same twelve chunks and the same rank composition. *cons.* is the fraction of names occurring in more than one chunk that every such chunk rendered identically; *real.* is the fraction of declared renderings that actually appear in the produced translation. One collective, an exclusive prefix scan costing four rounds, moves consistency by 0.409.

Arm	p	shared	consist.	cons.	real.
AMPI_Exscan	12	11	10	**0.909**	0.956
no coordination	12	12	6	**0.500**	0.957

7.3 Q2: translating a book with twelve agents

Workload. *Alice's Adventures in Wonderland* (Project Gutenberg #11, public domain), 141,230 characters and 25,922 words, partitioned into twelve balanced chunks (imbalance 1.10), translated into French. The book is close to embarrassingly parallel as prose but carries a dense set of coined proper nouns—the Mock Turtle, the Cheshire Cat, the Caucus-race—eleven of which occur in more than one chunk. Twelve independent translators will render them eleven different ways, and the result reads as though twelve people translated it, which they did.

Metric. Terminology consistency, computed without a judge model: for each name occurring in more than one chunk, do the chunks agree on the rendering? We report *declared* consistency (agreement between the glossaries the agents reported) and *realised* rate (whether the declared rendering actually appears in the produced translation), because a harness that measures only the first is measuring its own bookkeeping. Comparison is case- and accent-insensitive, so genuine lexical disagreements survive and typography does not create false ones.

Arms. The *no coordination* arm is the standard embarrassingly-parallel harness: broadcast the spec, scatter the chunks, translate, gather. The `AMPI_Exscan` arm inserts one collective: each rank proposes renderings for the names in its own chunk, an exclusive prefix scan with the `AMPI_FIRST` operator delivers the merged proposals of all *earlier* chunks, and each rank adopts the prefix where it exists. That is four extra rounds at $p = 12$ instead of the eleven a sequential chain would cost.

Result. The scan arm reached 0.909 declared consistency (10 of 11 shared terms rendered identically) against the baseline reported in Table 3, at a realised rate of 0.956 and 157,242 characters of French across twelve chunks. The single residual inconsistency, *the Rabbit-Hole*, split between *le Terrier du Lapin* and *le trou du Lapin*; it occurs in chunks 0 and 1, and chunk 0's rank was the one whose protocol violation we describe in Section 8.2, so its proposal never entered the prefix in time.

The collective cost was negligible against the agent cost, as the model predicts: broadcast, scatter and the scan completed in 0.14 s of the run's 3,051 s. Everything else was agents thinking. That ratio—one part in twenty thousand—is the empirical statement of $\gamma \gg \alpha, \beta$, and it is why the protocol can afford to be chatty.

The operator matters more than the collective. Before adopting `AMPI_FIRST` we ran the same harness with `AMPI_UNION`, the obvious choice. It scored 0.818 in a controlled replay against `AMPI_FIRST`'s 1.000 on identical inputs. Union is commutative, so when two chunks independently propose different renderings both survive into later prefixes and different ranks resolve them differently—the disagreement the scan exists to remove survives it. This is the concrete form of the principle in Section 4.4, and it cost us a run to find.

7.4 Q3: building a query engine with eight agents

Workload. `tinyq`, a columnar query engine over CSV supporting `SELECT`, `WHERE` with boolean operators and precedence, `GROUP BY` with five aggregates, `ORDER BY` and `LIMIT`, plus a command-line interface. Eight modules, one per rank, with a dependency graph of depth four: schema → csvio, lexer → parser → predicate, aggregate, and executor depending on four others.

This is the antithesis of the translation workload. Nothing is independent; the parser must emit precisely the tree the predicate evaluator consumes; the executor calls all six of the others; and no module can be validated in isolation against anything that matters.

Metric. A 24-test integration suite, written before the run and not editable by any rank, exercising every module and every named error case. Pass rate is the objective function. Because the suite is fixed and the interfaces are frozen in the broadcast specification, a module that is locally elegant and globally incompatible scores zero, which is the property we want.

Protocol. Broadcast the architecture; scatter the assignments; each rank publishes its realised interface to a shared window; *barrier*; each rank reads only its declared dependencies' interfaces via `AMPI_Win_get`; implement; *barrier*; rank 0 runs the suite and broadcasts the report; repair; *barrier*; rank 0 runs the suite again.

The two barriers are the experiment. Without the first, a rank reads an interface board that is half-written and invents the rest. Without the second, a rank is judged against a suite that most of its peers have not yet contributed to, and spends its one repair turn chasing failures that were never its own. Neither is expressible in an unstructured shared directory, which is what the comparison arm uses.

Table 4: Collaborative construction of `tinyq`: 8 mutually dependent modules, one per agent rank, 859 lines in total, judged by a frozen 24-test integration suite written before the run and editable by nobody. All 8 interfaces were published to the shared window at a total cost of 1994 tokens. The assembled package passes 24 of 24 tests (100%), with no interface mismatch between any pair of independently written modules.

Module (one agent rank each)	lines
`aggregate.py`	64
`cli.py`	47
`csvio.py`	62
`executor.py`	149
`lexer.py`	133
`parser.py`	242
`predicate.py`	59
`schema.py`	99

Result. The eight modules compose. The assembled package passes all 24 tests of the frozen integration suite, including every named error case, and there is *no interface mismatch between any pair of independently written modules*: the parser emits exactly the tuple shapes the predicate evaluator and the executor consume, the executor calls all six of its dependencies correctly, and the command-line interface drives the whole stack end to end. The eight interface declarations cost 1,994 tokens in total on the shared window, which is what each rank paid to know what its dependencies would provide — roughly 250 tokens per module against 859 lines of code produced.

We take that as the experiment's answer to Q3: publishing interfaces to a window behind a barrier, and reading only one's declared dependencies, is enough structure to make eight independently written modules of one program fit together on the first assembly. The comparison we would like to report—the same eight agents with an unstructured shared directory—is the one we could not run, for the reason below.

But the run did not complete as a run. The artifacts above are the agents' own work, produced under the protocol; the package was assembled and scored afterwards, because the run's collective control flow wedged before reaching its own measurement point. This is the more instructive half of the experiment, and it exposed a protocol bug the translation experiment had not. Rank 4's agent issued `ampi barrier`; its shell killed the command part way through, after the barrier's messages had been sent but before the process exited. Because the collective counter was only persisted at exit, the rank's next process reused the same number, replayed a barrier its peers had already completed, and was one collective behind them from then on. Every subsequent collective it issued carried a tag nobody was listening for, and the whole job wedged with all nine ranks blocked in a barrier.

Two things are notable. First, this is not the disobedience of Section 8.2: rank 4 followed its program exactly,

Table 5: Fault injection. Victim ranks stop without warning: no finalize, no error, no last heartbeat. *detect* is the median time from the failure to a survivor declaring it, against a 3 s liveness timeout. *ok* counts survivors that reached a defined outcome. Without recovery every survivor of every configuration fails; with revoke and shrink every survivor recovers, agrees on the same survivor set, and computes the correct result. Two repetitions per configuration.

p	dead	mode	detect (s)	ok	outcome
8	1	detect only	2.0	14/14	all detect
8	1	no recovery	25.1	0/7	**all fail**
8	1	revoke + shrink	2.0	14/14	**all recover**
8	2	detect only	2.0	12/12	all detect
8	2	no recovery	25.0	0/6	**all fail**
8	2	revoke + shrink	2.0	12/12	**all recover**
8	4	detect only	2.0	8/8	all detect
8	4	no recovery	25.0	0/4	**all fail**
8	4	revoke + shrink	2.0	8/8	**all recover**
16	1	detect only	2.0	30/30	all detect
16	1	no recovery	25.1	0/15	**all fail**
16	1	revoke + shrink	2.0	30/30	**all recover**
16	2	detect only	2.0	28/28	all detect
16	2	no recovery	25.1	0/14	**all fail**
16	2	revoke + shrink	2.0	28/28	**all recover**
16	4	detect only	2.0	24/24	all detect
16	4	no recovery	25.0	0/12	**all fail**
16	4	revoke + shrink	2.0	24/24	**all recover**

and the fault was ours. Second, our own collective-order verification did *not* catch it, because the mechanism compares operation *names* at a given sequence number, and a replayed barrier has the same name as the barrier its peers ran at that number. A rank that repeats a collective is indistinguishable, by that test, from a rank that is merely slow.

The fix is the write-ahead logging argument: the counter must be durable before the messages it labels, not after. `AMPI_Init` now persists the increment before the collective proceeds, at a cost of one small write per collective, and a regression test kills a rank mid-barrier and asserts that its next process issues collective 2 rather than collective 1. We report the failed run rather than only the repaired one because the failure is the evidence: a protocol whose counters are not durable will wedge exactly this way, silently, the first time an executor's shell times out.

7.5 Q4: surviving rank death

Ranks are killed for real: the executor stops mid-run, stops heartbeating, and says nothing. Survivors learn of it the only way anything learns of a crash, from the absence of further evidence. Executors are scripted here so that the configuration space can be covered with repetition; the corresponding evidence from real agents is in Section 8.2.

The result is unanimous across all eighteen configurations. Without recovery, *every* survivor of *every* configuration fails—the collective simply never completes, which is what an agent harness with no failure semantics does today, except that it hangs rather than returning an error. With detection enabled, every survivor identifies the failure within 2.0 s against a 3.0 s liveness timeout. With revoke and shrink, every survivor recovers, every survivor computes the *same* survivor set, and the subsequent collective over the shrunk communicator returns the correct result, at $p \in \{8, 16\}$ and with up to four simultaneous failures.

The consistency of the survivor set is the part worth emphasising. It is not enough that the survivors continue; they must continue *as a group*. Two ranks that disagreed about who died would build different communicators and deadlock on their next collective, which is why the shrink is an agreement and why it intersects rather than unions the proposals.

Building this experiment also found the bug described in Section 6: envelopes carried communicator-local ranks while the device addresses inboxes by world rank. On `AMPI_COMM_WORLD` the two numberings coincide, so the confusion survives every test that uses only the world communicator—and misroutes every message the moment a job shrinks. Before the fix, no collective on a shrunk communicator could complete.

7.6 What the trace shows

Because every operation is traced, we can report the structural quantities a harness author actually needs and cannot otherwise obtain. For the translation run: 13 ranks, a communication matrix dominated by the root's scatter and gather, a critical path of 4 collective rounds against 12 total agent turns, and a peak context pressure of 0.38 at the root—which is the gather, and which is the number that tells us the same harness would exhaust the root somewhere around 30 chunks. That prediction is exactly the allgather frontier of Figure 4 applied to a real run, and it is available before spending anything.

8 Discussion

8.1 A coordination failure we caused ourselves

The most instructive event in this work was not planned. Our launcher—the subagent API we used to start ranks—turned out to have a concurrency limit of ten. We asked for twelve translator ranks; nine started, three were refused, and the nine that started blocked in the exclusive scan waiting for peers that did not exist. Because every one of the nine held a launcher slot while blocked, no slot ever freed, and the three missing ranks could never start. The job was deadlocked by a resource limit two layers below the protocol, and no rank could observe the cause: each was correctly waiting for a message that a peer would have sent had it been allowed to run.

This is a textbook resource deadlock, and it is worth reporting for three reasons. First, it is exactly the class of failure a harness author does not anticipate, because the launcher's capacity is not part of the program. Second, it is invisible without a global view: `ampi peers` showed three ranks that had never checked in, which is the whole diagnosis, and nothing in any rank's local state would have revealed it. Third, the protocol already contains the correct response—`AMPI_Comm_shrink` over the ranks that did arrive, or a launch-time check that the launcher can supply p executors before the job starts—and we had implemented neither as a default. We now emit the second as a warning at `AMPI_Init` when the rank table is incomplete after the failure timeout.

The general lesson is that a collective's all-participants requirement makes the launcher's capacity a correctness property, not a performance one. MPI gets this for free because `mpiexec` either allocates p slots or fails. An agent launcher that partially fulfils a request produces a job that starts, looks healthy, and never finishes.

8.2 Agents disobey the protocol, and the protocol must survive it

We observed two protocol violations by agent ranks in a single run, and neither was a model quality problem in the usual sense.

Rank 1 decided that receiving its work assignment through `ampi scatter` was unnecessary. It read the corpus from disk instead and inferred which chunk was its own. It inferred wrongly—the scatter had assigned it chapter 1, and it took chapter 2—and it translated the wrong chapter competently and at length. Worse, skipping the collective left its collective counter one behind every peer's, so from then on it would have labelled every message with a tag nobody was listening for. This is the failure that motivated collective-order verification (Section 4); with that mechanism the run would have reported *"collective #2 was issued as 'scatterv' here and as 'exscan' by ranks [2,4]; this rank skipped a collective"* within seconds instead of hanging.

Rank 3 reordered its program, translating before running the scan it was supposed to run first, and then stopped without completing its remaining collectives. Its translation was good. Its participation was not.

Both are instances of what the MAST taxonomy calls disobeying the task specification [17], and both were locally reasonable: an agent that can see the corpus has no obvious reason to wait for a message containing part of it. The design conclusion we draw is that a protocol for agents cannot rely on participants following it, and must therefore make deviation *detectable* rather than merely forbidden. Contract checking does this for message content; collective-order verification does it for control flow; the failure lattice does it for liveness. We do not claim the set is complete.

There is a second, more uncomfortable conclusion. An agent that can reach the shared state by a path the protocol does not mediate will sometimes take it, and the protocol's guarantees evaporate silently when it does. MPI does not have this problem because a rank physically cannot read another address space. The nearest available remedy is to remove the side channel: run ranks in sandboxes whose only view of the job is the run directory. We did not do this, and we think a serious deployment should.

8.3 Three bugs, and what they have in common

The experiments found three defects in our own implementation, and they share a shape worth naming.

Sub-communicator misrouting. Envelopes carried communicator-local ranks; the device addresses inboxes by world rank. On `AMPI_COMM_WORLD` the two numberings coincide, so 260 tests passed and every collective on a shrunk or split communicator was misdelivered.

Non-durable collective counters. The counter separating one collective's traffic from the next was persisted at process exit. A rank killed mid-collective—an agent's shell timing out—reused the number, and was permanently one behind its peers.

Consumption at poll time. Our first matching engine marked a message consumed when it was polled rather than when it was matched, so a rank implemented as a sequence of short-lived commands destroyed every message it polled but did not match.

All three are the same mistake: an assumption that holds when a rank is one long-lived process on the world communicator, and fails when it is a succession of short-lived processes on a communicator that can change. The generalisable lesson is that *the endpoint being stateless is a load bearing property of an agent protocol*, not an implementation convenience, and every piece of protocol state has to be examined against it. A test suite that only exercises `AMPI_COMM_WORLD` with in-process ranks will not find any of them.

The second bug also exposes a limit of our own diagnostics. Collective-order verification compares operation *names* at a sequence number, so it detects a rank that skipped a collective and not a rank that repeated one: a replayed barrier has the same name as the barrier its peers ran. Detecting repetition would need a per-invocation nonce that all ranks derive identically, which we have not designed.

8.4 Operator algebra deserves more attention than it gets

The result we would most like other designers to take away is the one about commutativity (Section 4.4). Every framework that merges contributions from several agents is applying a reduction operator, and almost none of them ask whether the operator is associative or commutative.

The consequences are not subtle: our own first implementation of a shared-terminology scan used a union, which is commutative, and it left the participants disagreeing about a decision the scan existed to make. The error was invisible in small runs and appeared as an 18-point quality drop at twelve ranks.

Commutativity is the wrong default for consensus, and precedence is inexpressible without a rank order. We suspect this generalises well beyond terminology to any convention an agent team must settle—schema shapes, naming, file layout, API contracts—and that a good deal of reported multi-agent inconsistency is this bug.

8.5 Limitations

Scale. Our agent runs are at 12 ranks and our software build at 8. The microbenchmarks reach 128 ranks, but with scripted executors: they measure the protocol, not the agents. We cannot claim from this data that agent *quality* holds at 128 ranks, and we would expect it not to for allgather-shaped workloads for the reasons in Section 5.

One model family. All agent ranks were the same class of coding agent. AgentMPI's heterogeneity machinery—`RankSpec`, `AMPI_Comm_split_type` by model, cost-aware selection—is implemented and tested but not exercised against genuinely different models.

Statistical power. We report single runs of each configuration. Agent runs are expensive and slow, and turn durations are heavy-tailed, so the wall-clock numbers in particular should be read as one draw rather than as an estimate of a mean. The round counts are exact and deterministic; the consistency metrics are deterministic given the outputs; only the timings are noisy.

Operator intervention. Two ranks in the glossary run required manual completion of protocol steps they had skipped (Section 8.2). The translations they contributed are their own work, but the run is not a clean unattended execution, and we report it as such.

The failure detector is weak for shell-mode ranks. A rank implemented as a sequence of short-lived commands cannot heartbeat while it thinks, so its liveness timeout must exceed its longest turn. In our runs that meant a 30-minute timeout, which is a long time to wait to discover that an agent has died. A supervisor process per rank would fix this and is the obvious next step.

Not evaluated against alternative frameworks. We compare against an unstructured baseline, not against LangGraph or AutoGen. A fair comparison would require reimplementing the same workload in each, and the result would largely measure how much of the protocol those frameworks oblige the author to rebuild by hand—which is our claim, but not one this paper's data establishes.

9 Related work

9.1 Multi-agent LLM frameworks

The frameworks differ mainly in their concurrency model. AutoGen's v0.4 rewrite moved to an actor model with typed messages and a runtime that can be embedded or distributed, which is the closest of the major frameworks to a systems substrate [45, 44]. LangGraph compiles a graph over a shared, checkpointed state object; communication is by state mutation, and its `Send` API allows dynamic fan-out [40]. CrewAI and MetaGPT organise agents by role and pass work along a fixed pipeline [21, 35]. The OpenAI Agents SDK formalises the handoff: control, and the conversation, transfer wholesale [47]. AgentScope is unusual in explicitly adopting distributed-systems machinery (actors, an RPC layer) [2].

What none of them provides is a group abstraction with collective operations, a consistency model for shared state, or defined semantics under partial failure. Where a framework has a shared state object it is a blackboard without epochs; where it has fan-out it is a scatter without a matching gather contract; where it has retries it has no notion of what a peer already observed. Our claim is not that these frameworks are wrong, but that they are *applications* of a layer that does not exist, and therefore each rebuilds part of it incompatibly. That is the pre-MPI situation exactly.

9.2 Agent interoperability protocols

MCP standardises the client–server relationship between a model and its tools, resources and prompts, with a defined transport and lifecycle; A2A standardises agent-to-agent delegation with agent cards, tasks and artifacts; ACP and ANP occupy neighbouring positions [46, 1, 36, 27, 27]. All are valuable and all are orthogonal to this work: they specify how *two* parties exchange a request and a response, and AgentMPI specifies how p parties behave as a group. Nothing in MCP or A2A defines a barrier, a reduction, group membership, an epoch, or what happens to the other $p-1$ participants when one dies. Table 6 makes the comparison explicit.

9.3 Classical parallel and distributed models

The actor model [3] and its descendants—Erlang/OTP, Akka, Orleans [8, 41, 11]—supply asynchronous message passing, location transparency, and supervision. We take Erlang's supervision strategies directly, and its "let it crash" argument is the right default for agents for the reason Armstrong gave: defensive code for every failure mode is larger, buggier and less complete than a clean restart. What actors do not supply is collective operations or a group-scoped consistency model, which is precisely the gap MPI filled relative to the message-passing libraries of its own day.

Table 6: Agent protocols by layer. AGENTMPI is complementary to, not a replacement for, MCP and A2A: a rank may well use MCP to reach its tools.

	MCP	A2A	AgentMPI
Relationship	client–server	peer–peer	group
Unit	tool call	task	message, collective
Naming	server URI	agent card	rank in a communicator
Collectives	no	no	yes
Group membership	no	no	yes
Barrier / epochs	no	no	yes
Shared state model	resources	artifacts	window, defined epochs
Failure semantics	transport error	task state	failure lattice, ULFM
Context budget	no	no	first class

PGAS languages (UPC, Coarray Fortran, OpenSHMEM, Chapel, X10) [5] and task-based runtimes (Charm++, Legion, HPX) [20] offer a global address space or overdecomposition with dynamic load balancing. Charm++'s measurement-based load balancing is directly applicable to agents, whose per-task costs vary by an order of magnitude and are not predictable in advance; we do not implement it and regard it as clear future work.

Tuple spaces and the blackboard architecture are the intellectual ancestors of every agent scratchpad. Linda's tuple space is an associative shared memory with atomic `in`/`out`; Hearsay-II's blackboard is a shared structure that independent knowledge sources read and write [29]. Our `AMPI_Win_query` is closer to the Linda `rd` operation than to `MPI_Get`, and the resemblance is not accidental: an associative read is what a reader smaller than the memory needs.

KQML and FIPA-ACL are the historical precedent that should give the current protocol efforts pause. Both standardised agent communication in terms of speech acts and interaction protocols, both were technically careful, and neither achieved broad adoption [27]. Our reading is that they standardised the *semantics of intent*, which is the hardest thing to agree on and the least useful thing to fix, while leaving the mechanics to implementations. MPI did the opposite and succeeded. AGENTMPI follows MPI: it says nothing about what a message means.

9.4 Empirical studies of multi-agent failure

Cemri et al. analyse 200+ traces across seven multi-agent frameworks and derive the MAST taxonomy, in which the majority of failures are specification and inter-agent misalignment rather than reasoning errors [17]. Their categories map onto protocol features with uncomfortable directness: "disobey task specification" is a contract violation; "step repetition" and "loss of conversation history" are context management; "no or incomplete verification" is a missing validation contract; "premature termination" is a missing barrier. We regard that mapping as the strongest available evidence for this paper's thesis, and also as a caution: a protocol can make these failures *detectable*, and cannot make them impossible.

The practitioner literature is split, usefully. Cognition argues against multi-agent architectures on the grounds that context does not propagate reliably and implicit decisions conflict [56]; Anthropic reports a production multi-agent research system whose engineering difficulties were coordination, state and recovery [7]. Both diagnoses are the same. They differ on whether the problems are inherent. We think they are the ordinary problems of distributed systems, and that the field's tools for them are forty years old.

9.5 Serving and scheduling

The inference-serving layer is AGENTMPI's network fabric, and its properties matter. Continuous batching, paged attention and prefix caching [57, 39, 58] determine whether two ranks that share a prompt prefix share work, which is the agent analogue of two MPI ranks sharing a cache line. Our `AMPI_Comm_split_type` by model or provider exposes exactly this locality to the harness, and exploiting it properly—scheduling a reduction tree so that siblings share a prefix—is future work we find promising.

10 Conclusion

We set out to test how far MPI's design transfers to multi-agent systems built on language models, and the answer is: nearly all of it, and the places where it does not are the interesting ones.

What transfers is the load-bearing structure. Communicators with isolated contexts solve the library-composition problem for agent harnesses exactly as they solved it for parallel libraries in 1993, and multi-agent frameworks today exhibit the same silent interference that motivated them. The collective catalogue, the matching rules, the group algebra, the epoch structure of one-sided operations, two-phase collective I/O, the revoke/shrink/agree recovery vocabulary, and the name-shifted profiling interface all carry over with their semantics intact.

What does not transfer follows from four properties of the executor. Context is consumed rather than occupied, which turns a receive into an irreversible allocation, makes collective *feasibility* a question that must be answered before cost, and—as we found the hard way—means a reduction tree must be planned against the root's cumulative ingest rather than its per-round ingest. Reduction operators are semantic, so they must declare their algebra and the runtime must refuse the schedules that algebra does not license; the sharpest instance is that a commutative merge cannot express precedence and is therefore unsound for any collective whose purpose is agreement. Failure is a lattice rather than a binary, requiring independent liveness and progress signals and a content-level error class. And because computation costs five to seven orders of magnitude more than communication, latency-optimal

schedules win everywhere, which inverts several of MPI's standard algorithm choices.

Our evaluation is small in scale and honest about it. Twelve agents translating a book, eight building a query engine, and a fault-injection study are not a proof that this design scales to a hundred; the microbenchmarks that do reach 128 ranks measure the protocol rather than the agents. But the runs were real, they surfaced failures we did not anticipate—a launcher concurrency limit that deadlocked a collective, two agents that skipped protocol steps and reasoned confidently from inferred inputs—and each of those failures had a name and a remedy in the distributed-systems literature, which is the point.

The broader claim we would defend is a negative one. The current generation of agent protocols standardises the relationship between two parties, and the current generation of agent frameworks each rebuilds group semantics privately and incompatibly. That is the situation distributed-memory computing was in before 1994, and it was resolved not by a better framework but by a specification thin enough to implement many ways and precise enough to build libraries on. We think the same move is available here, and we have tried to show what it would look like.

Availability

The protocol specification, reference implementation, test suite, experiment harnesses, raw run data and the source of this paper are in the accompanying repository.

References

[1] A2A Project. What's new in v1.0. a2a-protocol.org, 2026. Cited as [A2A Docs 2026a]. Source for a2a.proto as normative, supportedInterfaces[], Signed Agent Cards (RFC 8785), multi-tenancy, the A2A-Version header, and the versioned extension mechanism.

[2] AgentScope. What's AgentScope 2.0? harness architecture and agent service deep dive. docs.agentscope.io, 2026. Cited as [AgentScope 2.0 Docs 2026]. Source for the stateless ReActAgent kernel, HarnessAgent, and externalized state stores.

[3] Gul Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, Cambridge, MA, 1986.

[4] Albert Alexandrov, Mihai F. Ionescu, Klaus E. Schauser, and Chris Scheiman. LogGP: Incorporating long messages into the LogP model — one step closer towards a realistic model for parallel computation. In *Proceedings of the 7th Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA '95)*, pages 95–105, Santa Barbara, CA, USA, July 1995. ACM. Also published as University of California, Santa Barbara Technical Report TRCS95-09.

[5] Michail Alvanos. *Optimization Techniques for Fine-Grained Communication in PGAS Environments*. PhD thesis, Universitat Politècnica de Catalunya, 2013.

[6] Lorenzo Alvisi and Keith Marzullo. Message logging: Pessimistic, optimistic, and causal. In *Proceedings of the 15th International Conference on Distributed Computing Systems (ICDCS)*, pages 229–236. IEEE Computer Society, 1995.

[7] Anthropic. How we built our multi-agent research system. Anthropic Engineering Blog, 2025. Cited as [Anthropic 2025]. Source for the 90.2% internal-eval improvement, token usage explaining 80% of BrowseComp variance, the 15x token multiplier, and the explicit statement that domains requiring shared context or many inter-agent dependencies are a poor fit.

[8] Joe Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, Royal Institute of Technology (KTH), Stockholm, Sweden, 2003.

[9] Pavan Balaji, William Gropp, Ewing Lusk, and Rajeev Thakur. Translational research in the MPICH project. *Journal of Computational Science*, 2020. UNVERIFIED: complete author list and volume/pages; the DOI and title are confirmed.

[10] David E. Bernholdt, Swen Boehm, George Bosilca, Manjunath Gorentla Venkata, Ryan E. Grant, Thomas Naughton, Howard P. Pritchard, Martin Schulz, and Geoffroy R. Vallée. A survey of MPI usage in the US exascale computing project. *Concurrency and Computation: Practice and Experience*, 32(3):e4851, 2020.

[11] Philip A. Bernstein, Sergey Bykov, Alan Geller, Gabriel Kliot, and Jorgen Thelin. Orleans: Distributed virtual actors for programmability and scalability. Technical Report MSR-TR-2014-41, Microsoft Research, 2014. Cited as [Bernstein et al. 2014]. Source for the four facets of virtual actors and for the Akka comparison (at-most-once delivery, per-pair FIFO, supervision hierarchy, location fixed at creation). Contains the quotation about re-instantiating an actor on another server after a crash.

[12] Wesley Bland, Aurelien Bouteiller, Thomas Herault, George Bosilca, and Jack Dongarra. Post-failure recovery of MPI communication capability: Design and rationale. *The International Journal of High Performance Computing Applications*, 27(3):244–254, 2013.

[13] Wesley Bland, Aurélien Bouteiller, Thomas Herault, Joshua Hursey, George Bosilca, and Jack J. Dongarra. An evaluation of user-level failure mitigation support in MPI. In *Recent Advances in the Message Passing Interface: 19th European MPI Users' Group Meeting (EuroMPI 2012), Vienna, Austria, September 23–26, 2012*, Lecture Notes in Computer Science, pages 193–203. Springer, 2012.

[14] Luc Bomans, Dirk Roose, and Rolf Hempel. The Argonne/GMD macros in FORTRAN for portable parallel programming and their implementation on the Intel iPSC/2. *Parallel Computing*, 15(1–3):119–132, 1990.

[15] Ralph M. Butler and Ewing L. Lusk. Monitors, messages, and clusters: The p4 parallel programming system. *Parallel Computing*, 20(4):547–564, 1994.

[16] Ralph H. Castain, David Solt, Joshua Hursey, and Aurelien Bouteiller. PMIx: Process management for exascale environments. In *Proceedings of the 24th European MPI Users' Group Meeting (EuroMPI)*, 2017.

[17] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025. Cited as [Cemri et al. 2025]. The MAST taxonomy: 14 failure modes in 3 categories (Specification 41.8%, Inter-Agent Misalignment 36.9%, Task Verification 21.3%). Grounded Theory over traces averaging >15,000 lines

from 7 open-source MAS frameworks; six annotators; Cohen's kappa = 0.88; MAST-Data has 1600+ annotated traces. Also the source of Insight 2 (protocol-level fixes are often insufficient for FC2 failures, which demand deeper social reasoning) and of the intervention results (+9.4% from improved role specification, +15.6% from a task-objective verification step on ProgramDev). Artifacts: `https://github.com/multi-agent-systems-failure-taxonomy/MAST`.

[18] Ernie Chan, Marcel Heimlich, Avi Purkayastha, and Robert van de Geijn. Collective communication: Theory, practice, and experience. *Concurrency and Computation: Practice and Experience*, 19(13):1749–1783, 2007. Preprint: FLAME Working Note #22, The University of Texas at Austin, Department of Computer Sciences, Technical Report TR-06-44, September 2006.

[19] K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985.

[20] Charm++ Development Team. *The Charm++ Parallel Programming System Manual*. Parallel Programming Laboratory, University of Illinois at Urbana-Champaign, 2025. `https://charm.readthedocs.io/`.

[21] CrewAI. Hierarchical process; processes. docs.crewai.com, 2026. Cited as [CrewAI Docs 2026]. Source for Process.sequential/hierarchical and manager_llm.

[22] David E. Culler, Richard M. Karp, David A. Patterson, Abhijit Sahay, Klaus E. Schauser, Eunice Santos, Ramesh Subramonian, and Thorsten von Eicken. LogP: Towards a realistic model of parallel computation. In *Proceedings of the 4th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '93)*, pages 1–12. ACM, 1993. Journal version: Culler et al., "LogP: A Practical Model of Parallel Computation," Communications of the ACM 39(11):78–85, 1996, doi:10.1145/240455.240477.

[23] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, February 2013.

[24] James Dinan, Pavan Balaji, Darius Buntinas, David Goodell, William Gropp, and Rajeev Thakur. An implementation and evaluation of the MPI 3.0 one-sided communication interface. *Concurrency and Computation: Practice and Experience*, 2016.

[25] Jack J. Dongarra, Rolf Hempel, Anthony J. G. Hey, and David W. Walker. A proposal for a user-level, message-passing interface in a distributed memory environment. Technical Report ORNL/TM-12231, Oak Ridge National Laboratory, February 1993. Revision of the "MPI1" preliminary draft first circulated in November 1992.

[26] Jack J. Dongarra, Steve W. Otto, Marc Snir, and David Walker. A message passing standard for MPP and workstations. *Communications of the ACM*, 39(7):84–90, July 1996.

[27] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). *arXiv preprint arXiv:2505.02279*, 2025. Cited as [Ehtesham et al. 2025]. Note for the dossier's argument: this survey's comparison axes are interaction modes, discovery, communication patterns and security — it has no dimension for collectives, membership or consistency.

[28] E. N. (Mootaz) Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, 34(3):375–408, 2002.

[29] Lee D. Erman, Frederick Hayes-Roth, Victor R. Lesser, and D. Raj Reddy. The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty. *ACM Computing Surveys*, 12(2):213–253, 1980. Cited as [Erman et al. 1980]. Canonical blackboard architecture: diverse, independent, asynchronous knowledge sources cooperating by data-directed invocation over a shared database.

[30] Graham E. Fagg and Jack Dongarra. FT-MPI: Fault tolerant MPI, supporting dynamic applications in a dynamic world. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface (EuroPVM/MPI 2000)*, volume 1908 of *Lecture Notes in Computer Science*, pages 346–353. Springer, 2000.

[31] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.

[32] Al Geist, Adam Beguelin, Jack Dongarra, Weicheng Jiang, Robert Manchek, and Vaidy Sunderam. *PVM: Parallel Virtual Machine — A Users' Guide and Tutorial for Networked Parallel Computing*. MIT Press, 1994. History chapter says PVM 2 was "released in March 1991"; the version appendix lists PVM 2.0 (Feb. 1991) and 2.1 (Mar. 1991).

[33] William D. Gropp. Learning from the success of MPI. In *High Performance Computing — HiPC 2001: 8th International Conference, Hyderabad, India, December 17–20, 2001*, volume 2228 of *Lecture*

Notes in Computer Science, pages 81–92. Springer, December 2001. Delivered as the keynote "Whither MPI: Lessons From and the Future of MPI". Springer gives pp. 81–92; DBLP gives pp. 81–94. Preprint: arXiv:cs/0109017, doi:10.48550/arXiv.cs/0109017.

[34] Roger W. Hockney. The communication challenge for MPP: Intel Paragon and Meiko CS-2. *Parallel Computing*, 20(3):389–398, March 1994.

[35] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024. Cited as [Hong et al. 2024]. Source of the shared message pool and subscription-mechanism quotations.

[36] IBM. What is Agent Communication Protocol (ACP)? ibm.com/think/topics/agent-communication-protocol, 2026. Cited as [IBM 2026]. Now carries a migration notice following the A2A merger.

[37] Tanzima Z. Islam, Kathryn Mohror, and Martin Schulz. Exploring the capabilities of the new MPI_T interface. In *Proceedings of the 21st European MPI Users' Group Meeting (EuroMPI/ASIA)*, 2014.

[38] Andreas Knüpfer, Christian Rössel, Dieter an Mey, Scott Biersdorff, Kai Diethelm, Dominic Eschweiler, Markus Geimer, Michael Gerndt, Daniel Lorenz, Allen D. Malony, Wolfgang E. Nagel, Yury Oleynik, Peter Philippen, Pavel Saviankou, Dirk Schmidl, Sameer Shende, Ronny Tschüter, Michael Wagner, Bert Wesarg, and Felix Wolf. Score-p: A joint performance measurement run-time infrastructure for periscope, scalasca, tau, and vampir. In *Tools for High Performance Computing 2011*, pages 79–91. Springer, 2012.

[39] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*, 2023. Cited as [Kwon et al. 2023]. vLLM.

[40] LangChain. Persistence: Checkpointers vs. stores; subgraph checkpoint namespaces. docs.langchain.com (LangGraph), 2026. Cited as [LangChain Docs 2026b]. Source for "each subgraph manages its own checkpoint namespace" and the Store workaround.

[41] Lightbend, Inc. and Apache Software Foundation. Phi accrual failure detector: Akka/Apache Pekko cluster documentation and implementation. `https://doc.akka.io/libraries/akka-core/current/typed/failure-detector.html`, 2025. Accessed 2026.

[42] Message Passing Interface Forum. MPI: A message-passing interface standard. *International Journal of Supercomputer Applications and High Performance Computing*, 8(3/4):165–414, 1994. Special Issue on MPI. The corresponding standard document is dated 5 May 1994.

[43] Message Passing Interface Forum. MPI-2: Extensions to the message-passing interface. Technical report, University of Tennessee, Knoxville, July 1997. MPI-2.0 document date 18 July 1997; also contains MPI-1.2.

[44] Microsoft. AutoGen Core: Agent and agent runtime; message and communication. AutoGen documentation, 2025. Cited as [AutoGen Core Docs 2025]. Source for AgentId, TopicId, TypeSubscription, and the statement that exceptions raised while handling published messages are logged but not propagated to the publisher.

[45] Microsoft Research. AutoGen v0.4: Reimagining the foundation of agentic AI for scale, extensibility, and robustness. Microsoft Research Blog, 2025. Cited as [Microsoft 2025a]. Source of the "we ended up adopting an actor model" quotation.

[46] Model Context Protocol. The 2026-07-28 specification. blog.modelcontextprotocol.io, July 2026. Cited as [MCP Blog 2026]. Primary source for the statelessness overhaul: removal of the initialize handshake and Mcp-Session-Id (SEP-2575, SEP-2567), mandatory server/discover, Mcp-Method and Mcp-Name routing headers (SEP-2243), deprecation of sampling/elicitation/roots in favour of Multi Round-Trip Requests (SEP-2577), and the subscriptions/listen and tasks/* changes.

[47] OpenAI. Handoffs. OpenAI Agents SDK documentation, 2026. Cited as [OpenAI Agents Docs 2026a]. Source for input_filter/on_handoff, "handoffs stay within a single run", and asymmetric guardrail placement.

[48] OpenBMB/ChatDev. What's the difference between ChatDev and MetaGPT? GitHub Issue #24, 2023. Cited as [ChatDev Issue 2023]. Source of the phase-level-passing vs. agent-level-broadcasting comparison.

[49] Pitch Patarasuk and Xin Yuan. Bandwidth optimal all-reduce algorithms for clusters of workstations. *Journal of Parallel and Distributed Computing*, 69(2):117–124, 2009.

[50] Rolf Rabenseifner. Optimization of collective reduction operations. In *Computational Science — ICCS 2004*, volume 3036 of *Lecture Notes in Computer Science*, pages 1–9, Berlin, Heidelberg, 2004. Springer.

[51] Sameer S. Shende and Allen D. Malony. The tau parallel performance system. *International Journal of High Performance Computing Applications*, 20(2):287–311, 2006.

[52] Anthony Skjellum, Steven G. Smith, Nathan E. Doss, Alvin P. Leung, and Manfred Morari. The design and evolution of Zipcode. *Parallel Computing*, 1994. Invited paper, Special Issue on Message Passing. Preprint dated 8 March 1994.

[53] Rajeev Thakur, William Gropp, and Ewing Lusk. Data sieving and collective I/O in ROMIO. In *Proceedings of the 7th Symposium on the Frontiers of Massively Parallel Computation (Frontiers '99)*, pages 182–189. IEEE Computer Society, February 1999. Also Argonne National Laboratory Preprint ANL/MCS-P723-0898.

[54] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. Optimization of collective communication operations in MPICH. *International Journal of High Performance Computing Applications*, 19(1):49–66, 2005.

[55] David W. Walker. Standards for message passing in a distributed memory environment. Technical Report ORNL/TM-12147, Oak Ridge National Laboratory, August 1992. Report of the First CRPC Workshop on Standards for Message Passing in a Distributed Memory Environment, 29–30 April 1992, Williamsburg, VA; 68 attendees. OSTI ID 10170156.

[56] Walden Yan. Don't build multi-agents. Cognition blog, 2025. Cited as [Yan 2025]. Two principles: share full agent traces rather than only messages; actions carry implicit decisions, and conflicting decisions produce bad results.

[57] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*, 2022. Cited as [Yu et al. 2022]. Origin of iteration-level scheduling (continuous batching).

[58] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. Cited as [Zheng et al. 2024]. RadixAttention: variable-length longest-prefix KV reuse via a radix tree.